

Windows绘图

党宇

dangyu@nankai.edu.cn



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學

Nankai University

主要内容

- 主框架窗口与视图窗口
 - GDI与设备环境
 - 常用绘图函数
 - 绘图工具
 - 文本处理
 - 位图操作
 - 图标与光标
 - 综合实例
-
- 参考课本：第四、五章

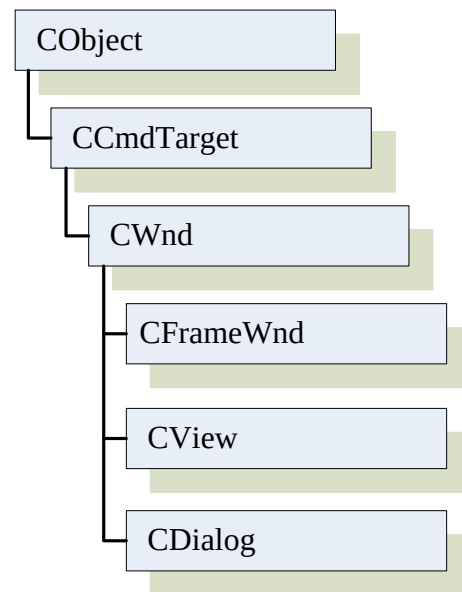
主框架窗口与视图窗口

➤ CFrameWnd

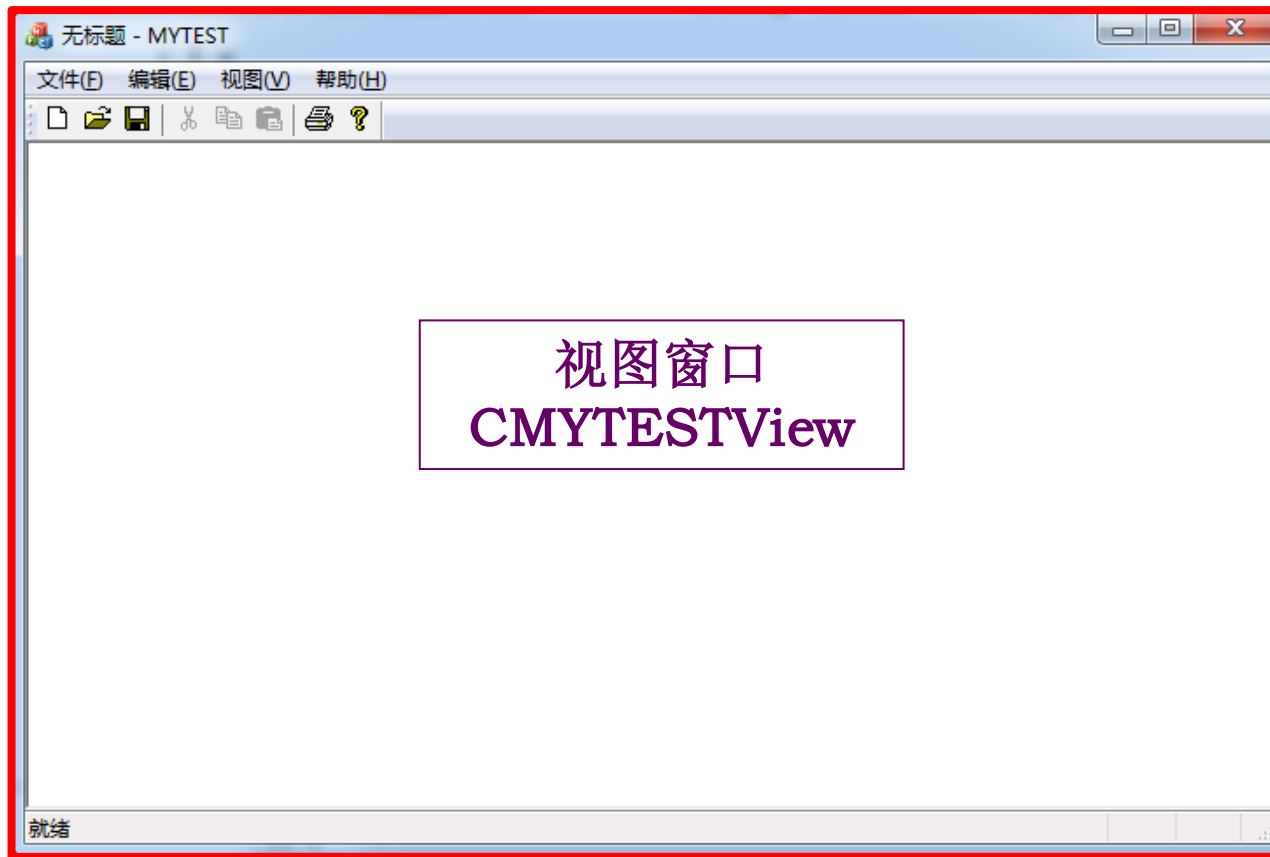
- MFC框架窗口类，是单文档框架窗口的基类
- 维护菜单、工具条、状态条的显示、更新，视图的位置和显示等

➤ CView

- MFC视图基类，实现框架窗口中的客户区
- **OnDraw()**: 在屏幕发生变化或焦点的变化需要重绘时调用
- 视图窗口覆盖在主框架窗口之上



主框架窗口与视图窗口



主框架窗口
CMainFrame

主框架窗口与视图窗口

- 举例1：在CMainFrame类和CMYTESTView类中分别响应鼠标左键单击消息

```
void CMYTESTView::OnLButtonDown(UINT nFlags, CPoint point)
{
    // TODO: 在此添加消息处理程序代码和/或调用默认值
    MessageBox(_T("left button down"));
    CView::OnLButtonDown(nFlags, point);
}
```

```
void CMainFrame::OnLButtonDown(UINT nFlags, CPoint point)
{
    // TODO: 在此添加消息处理程序代码和/或调用默认值
    MessageBox(_T("left button down"));
    CFrameWnd::OnLButtonDown(nFlags, point);
}
```

添加蓝色
代码

主框架窗口与视图窗口

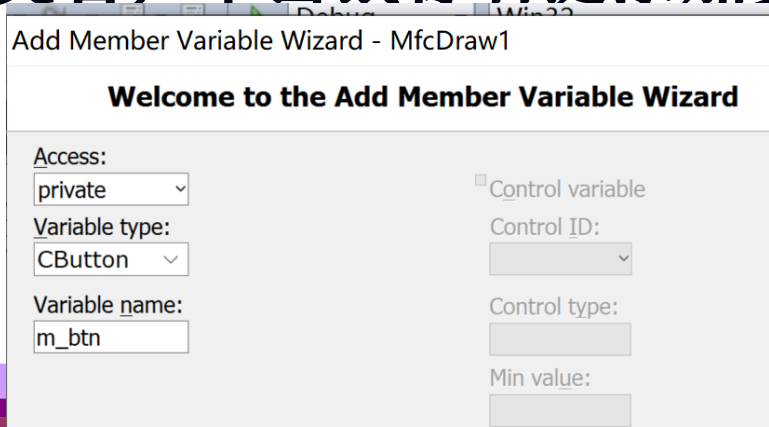
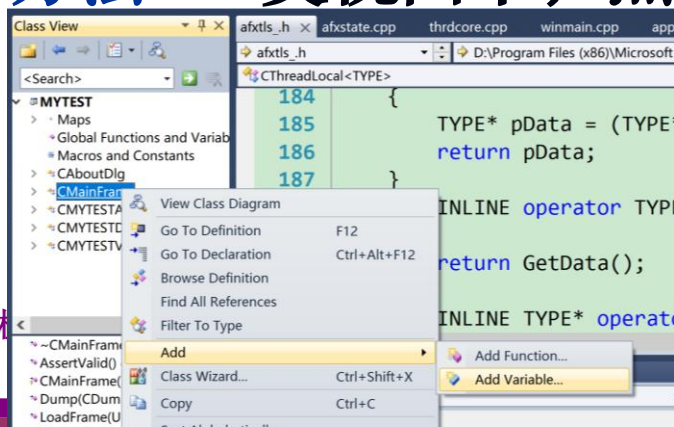
- 举例2：在CMainFrame类和CMYTESTView类中分别显示按钮

- **第一步** 在类定义中添加成员对象

```
CButton      m_btn;
```

- **方法1**：直接找到两个类的**头文件**，在类定义中添加上述代码。【资源视图】->【头文件】->【MainFrm.h和MYTESTView.h】 **方法3**：Ctrl+Shift+X

- **方法2**：类视图中，点击类名，单击鼠标右键添加变量



主框架窗口与视图窗口

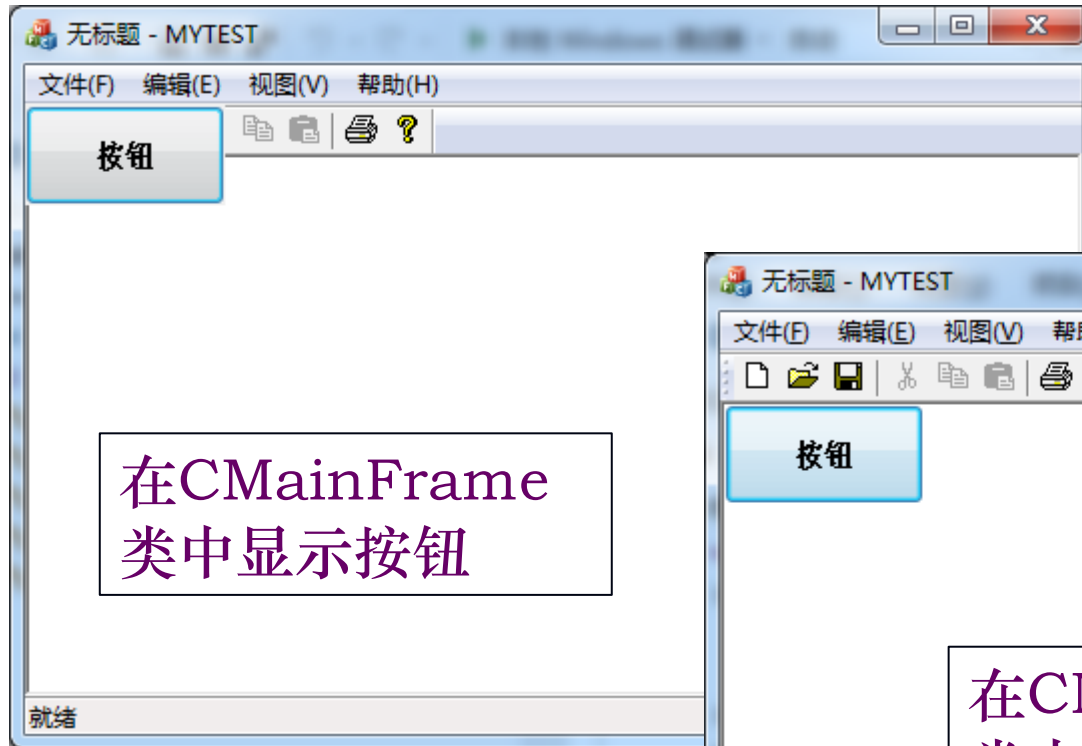
- 举例2：在CMainFrame类和CMYTESTView类中分别显示按钮
- 第二步，在两个类的OnCreate()中添加按钮

```
m_btn.Create(_T("按钮"),WS_CHILD |  
    BS_DEFPUSHBUTTON,CRect(0,0,200,100),this,123);  
m_btn.ShowWindow(SW_SHOWNORMAL);
```

*按钮的Create()函数原型

```
virtual BOOL Create(LPCTSTR lpszCaption, DWORD dwStyle,  
const RECT& rect, CWnd* pParentWnd, UINT nID);
```

主框架窗口与视图窗口



定制主框架窗口

➤ CREATESTRUCT结构用于设置窗口外观

```
typedef struct tagCREATESTRUCTW {  
    LPVOID      lpCreateParams;  
    HINSTANCE    hInstance;  
    HMENU        hMenu;  
    HWND         hwndParent;  
    int          cy;           // 窗口高度  
    int          cx;           // 窗口宽度  
    int          y;            // 窗口左上角y坐标  
    int          x;            // 窗口左上角x坐标  
    LONG         style;        // 窗口样式  
    LPCWSTR      lpszName;  
    LPCWSTR      lpszClass;  
    DWORD        dwExStyle;  
} CREATESTRUCT;
```

系统程序

定制主框架窗口

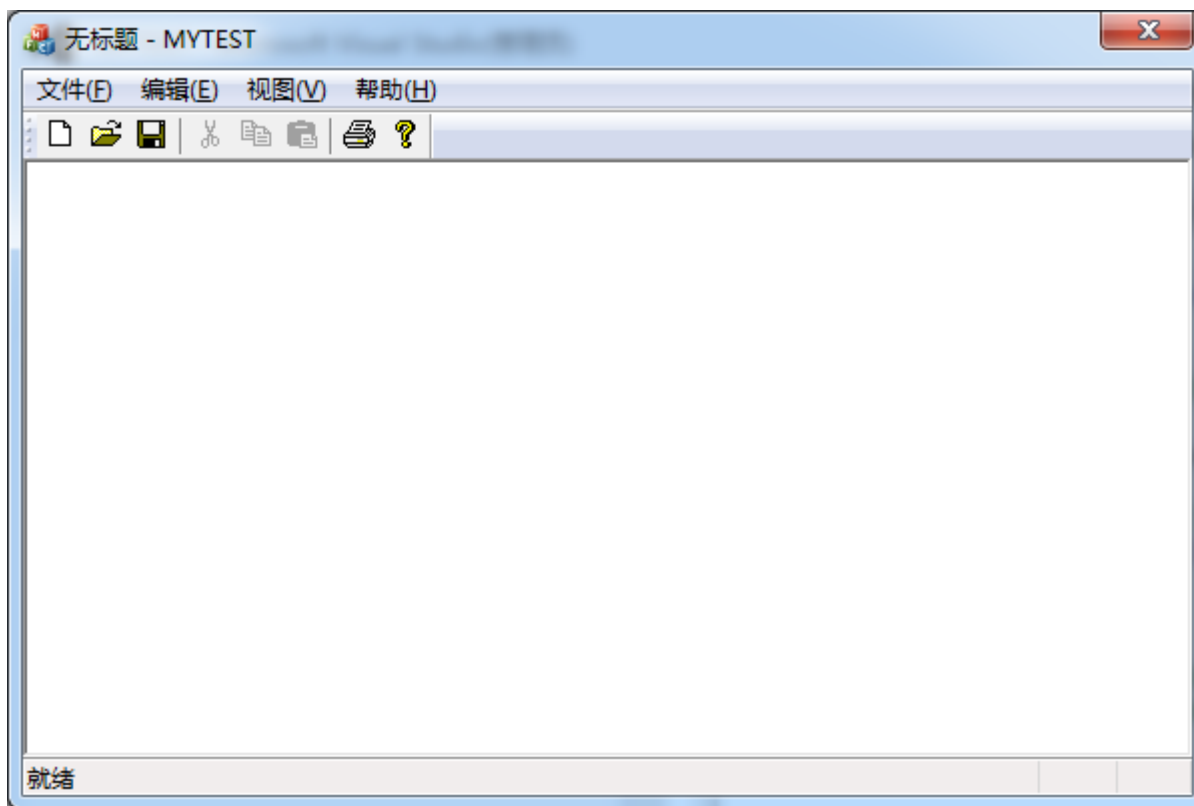
- 举例：在CMainFrame::PreCreateWindow()中添加代码，定制主框架窗口
- 通过位运算将多个样式定义成组合式的窗口样式

```
cs.cx=600;    // 设置窗口大小
cs.cy=400;
cs.x=0;       // 设置窗口位置
cs.y=0;

// 设定窗口大小不可变
cs.style &= ~WS_THICKFRAME;
// 取消最大最小化按钮
cs.style &= ~WS_MAXIMIZEBOX & ~WS_MINIMIZEBOX;
```

定制主框架窗口

- 举例：在 `CMainFrame::PreCreateWindow()` 中添加代码，定制主框架窗口

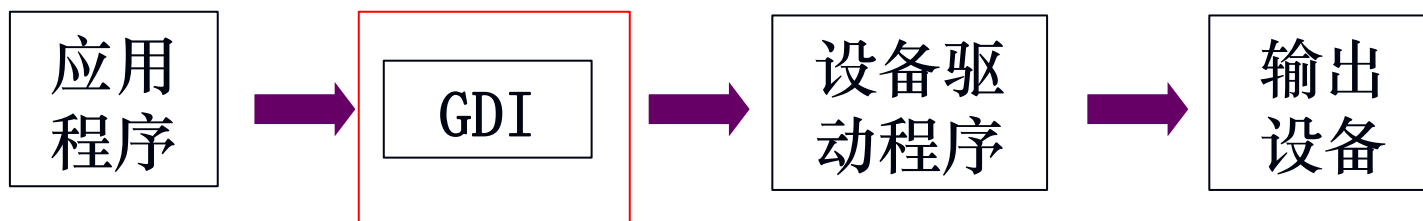


主要内容

- 主框架窗口与视图窗口
- GDI与设备环境
- 常用绘图函数
- 绘图工具
- 文本处理
- 位图操作
- 图标与光标
- 综合实例

GDI (图形设备接口)

- Windows应用程序使用图形设备接口(Graphics Device Interface, GDI)和Windows设备驱动程序支持与设备无关的图形
- GDI是抽象接口，作为Windows的重要组成部分，负责管理用户绘图操作时功能的转换
- 用户通过调用GDI 函数与设备打交道，GDI通过不同设备提供的驱动程序将绘图语句转换为对应的绘图指令，避免了用户对硬件直接进行操作，实现设备无关性（显示器或打印机）



GDI 的图形输出

➤ 矢量图形

- 画线和填充图形，包括点、直线、曲线、多边形、扇形和矩形等

➤ 光栅图形

- 通过光栅图形函数对以位图形式存储的数据进行操作，包括各种位图和图标输出。
- 屏幕：对若干行和列的像素操作
- 打印机：对若干行和列的点阵输出
- 直接从内存到显存的复制操作，速度快，内存要求高

➤ 文本

- 以图形方式输出文本，以逻辑坐标为单位计算输出位置，而DOS是以行为单位
- 用户可以设置文本的各种效果，如加粗、斜体、设置颜色等

DC (设备环境)

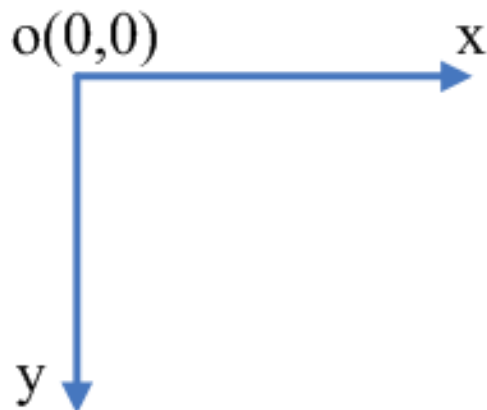
- 应用程序的输出面向设备环境(Device Context, DC)
- DC是一个虚拟逻辑设备，也称设备描述表或设备上下文
- GDI利用DC将各程序的输出限制在自己的窗口中
- DC定义了一系列图形对象及其属性的结构，用于程序的可视化输出，它提供了一张画布，可在上面书写文字，或绘制点、直线、曲线等
- DC的类型：显示类型、打印类型、存储类型、消息类型

DC (设备环境)

- DC是一种数据结构，它包含GDI需要的所有关于显示平面情况的描述字段，包括相连的物理设备和各种状态信息
- DC的主要功能
 - 允许应用程序使用一个输出设备
 - 提供应用程序、设备驱动和输出设备之间的连接
 - 保存当前信息，例如当前的画笔、画刷、字体和位图等图形对象及其属性，以及颜色和背景等影响图形输出的绘图模式
 - 保存窗口剪切区域(Clipping Region)，限制程序输出到输出设备中窗口覆盖的区域

设备坐标系

- 设备坐标系是Windows的各种不同类型的设备所使用的坐标系，属于笛卡尔坐标系
- 单位为像素，又称设备单位
- x轴自左至右，y轴从上到下，坐标原点在屏幕左上角



设备环境类

- MFC封装了DC，提供CDC 类及它的子类以访问GDI
- CDC：派生自CObject类， MFC的设备环境基类，封装所有GDI绘图函数
 - CPaintDC： OnPaint函数使用的设备环境类，用于响应窗口重绘(WM_PAINT)消息
 - CClientDC： 窗口客户区（窗口中除边框、标题栏、菜单栏、状态栏外的中间部分）的设备环境类，响应非窗口消息，并对客户区绘图时使用
 - CWindowDC： 程序窗口的设备环境类，包括客户区和非客户区
 - CMetaFileDC： Windows图元文件的设备环境类

设备环境类

➤ CPaintDC

- 当窗口的某个区域需要重绘时激发窗口重绘消息WM_PAINT，相应消息处理函数CWnd::OnPaint将被调用
- CPaintDC一般只用于OnPaint函数中，在完成窗口重绘后，CPaintDC对象的析构函数把WM_PAINT消息从消息队列中清除，避免不断重绘操作
- 坐标原点(0,0)是客户区的左上角

➤ CClientDC

- 用于特定窗口客户区的输出，其构造函数中包含了GetDC，析构函数中包含了ReleaseDC，不需要显式释放DC资源
- 一般用于响应非重绘消息（如键盘和鼠标消息）的绘图操作
- 坐标原点(0,0)是客户区的左上角

➤ CWindowDC

- 在整个应用程序窗口上画图，除非自己绘制窗口边框和按钮，否则一般不用
- 坐标原点(0,0)是屏幕的左上角

在MFC中获取DC

- 在一些函数中直接传递一个指向CDC 对象的指针

`OnDraw(CDC* pDC);`

- 使用构造函数构建对象

- 在CWnd类的OnPaint函数中，定义CPaintDC对象

`CPaintDC dc(...);`

- 在CWnd类的其它函数中，定义CClientDC和CWindowDC对象

`CClientDC dc(...);/CWindowDC dc(...);`

- 通过GetDC获得设备环境指针

`CDC* pDC=GetDC();`

`ReleaseDC(pDC);`

举例：利用不同的DC画直线

- 绘制直线的函数
 - MoveTo(当前位置)和 LineTo(画直线)
- 在C***View类中定义直线的起点和终点，并初始化

在C***View类定义中添加

private:

```
CPoint m_pStart,m_pEnd;
```

在C***View类构造函数中添加

```
m_pStart.x = 0;
```

```
m_pStart.y = 0;
```

```
m_pEnd.x = 200;
```

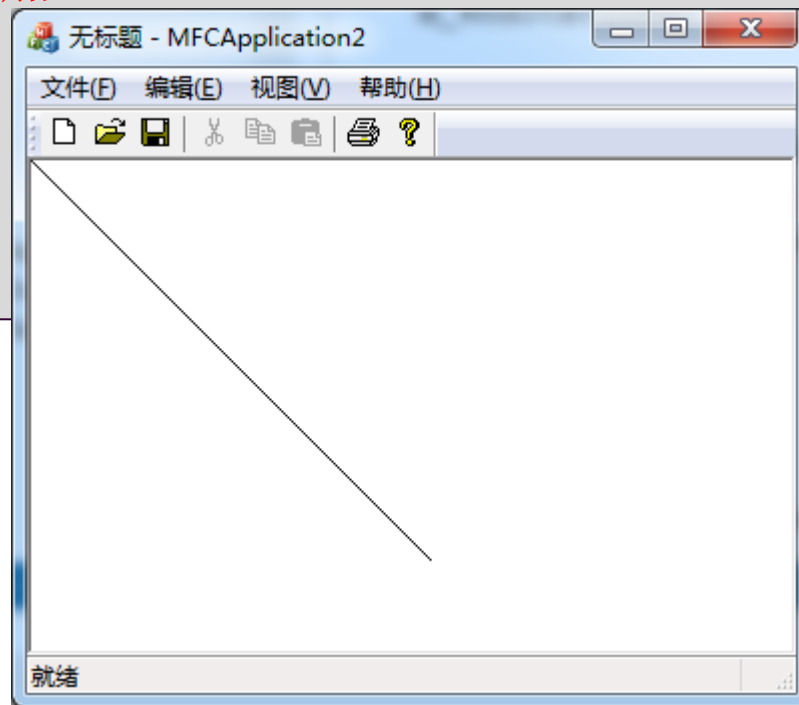
```
m_pEnd.y = 200;
```

举例：利用不同的DC画直线

- 响应鼠标右键单击消息WM_RBUTTONDOWN，在消息响应函数中添加代码
- 1、使用GetDC()获得客户区对象

在C***View类OnDraw()函数中添加

```
CDC *pDC=GetDC();  
pDC->MoveTo(m_pStart);  
pDC->LineTo(m_pEnd);  
ReleaseDC(pDC);
```



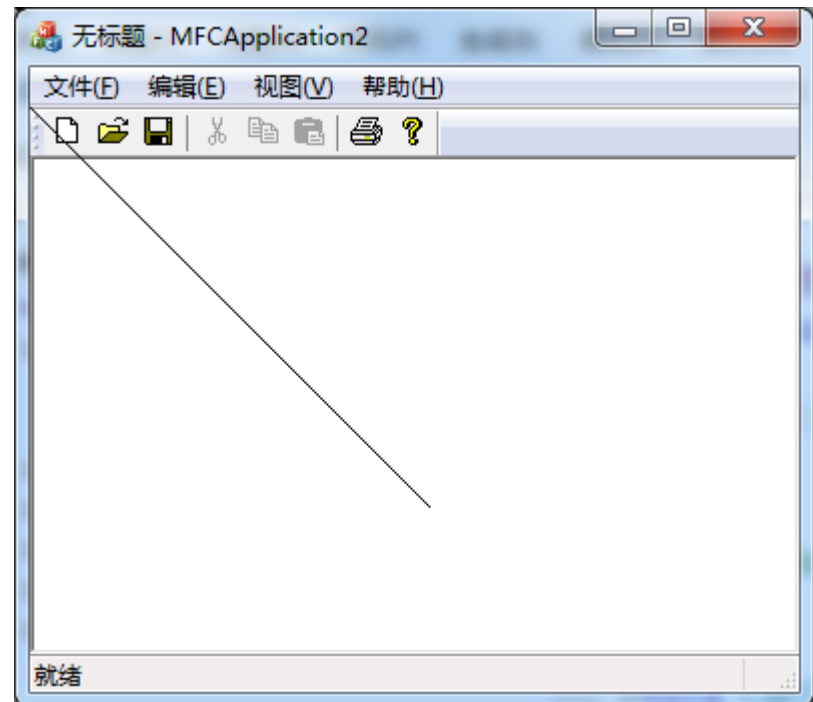
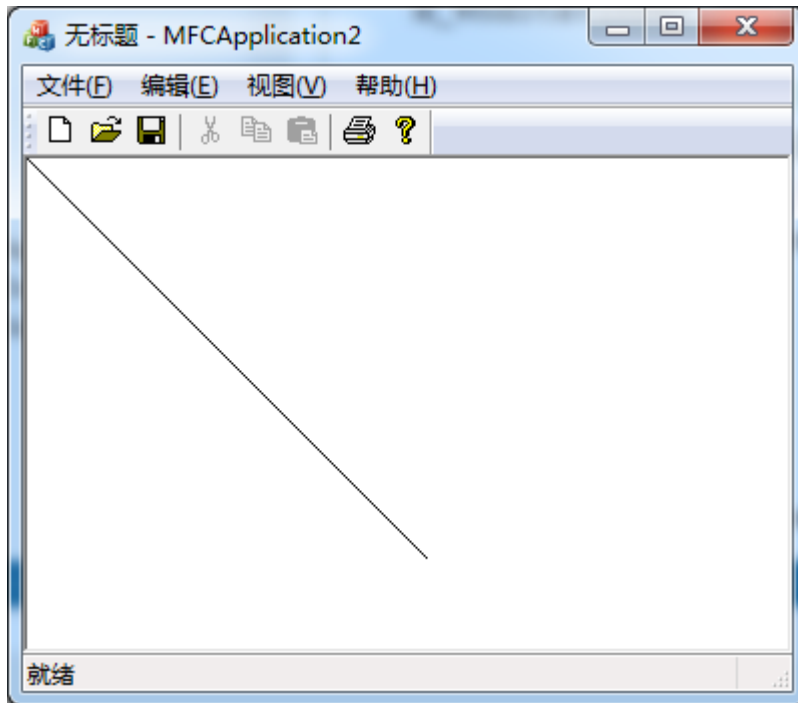
举例：利用不同的DC画直线

➤ 2、使用CClientDC类构造函数

OnDraw

```
CClientDC dc(this);  
dc.MoveTo(m_pStart);  
dc.LineTo(m_pEnd);
```

```
CClientDC dc(GetParent());  
dc.MoveTo(m_pStart);  
dc.LineTo(m_pEnd);
```





举例：利用不同的DC画直线

➤ 3、使用CWindowDC类构造函数

OnDraw

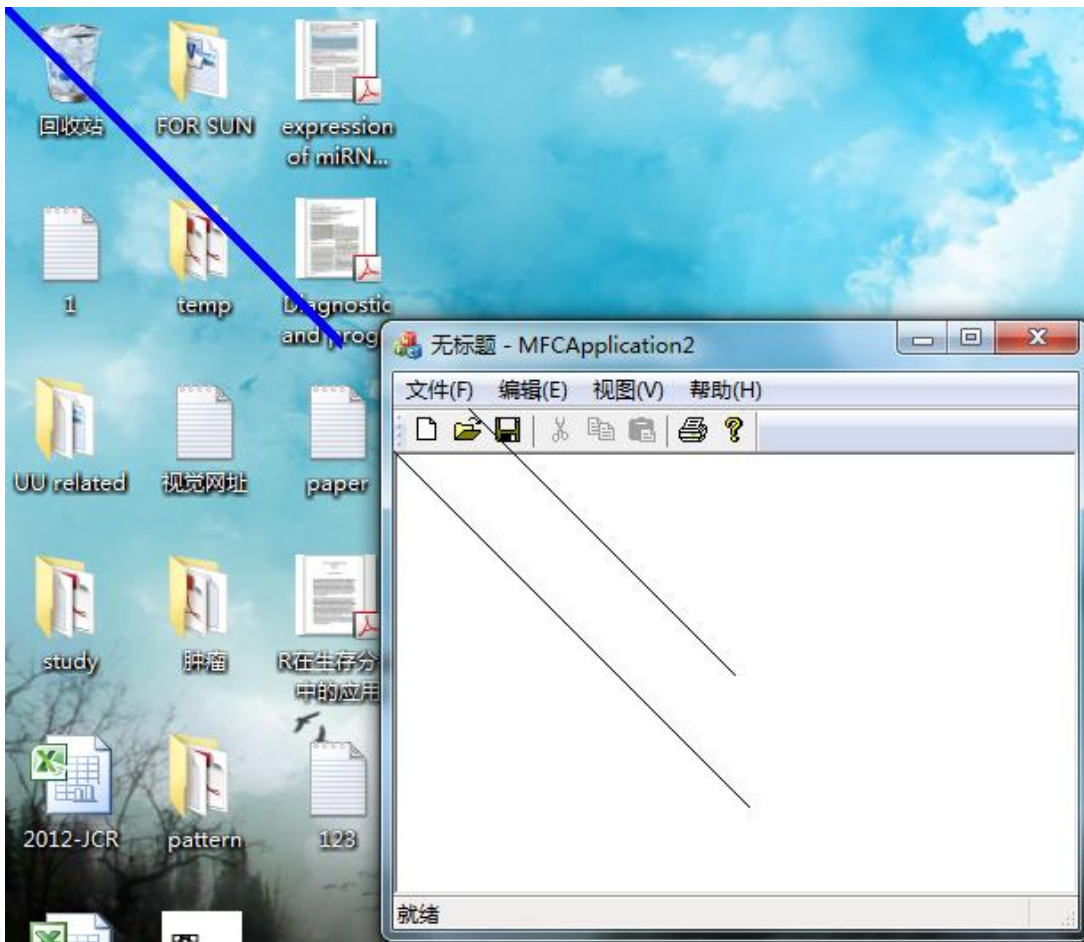
```
CWindowDC dc1(this);                // 客户区
dc1.MoveTo(m_pStart);
dc1.LineTo(m_pEnd);

CWindowDC dc2(GetParent());          // 整个窗口
dc2.MoveTo(m_pStart);
dc2.LineTo(m_pEnd);

CWindowDC dc3(GetDesktopWindow()); // 整个屏幕
CPen NewPen(PS_SOLID,5,RGB(0,0,255));
CPen *pOldPen;
pOldPen=dc3.SelectObject(&NewPen);
dc3.MoveTo(m_pStart);
dc3.LineTo(m_pEnd);
dc3.SelectObject(pOldPen);
```


举例：利用不同的DC画直线

➤ 3、使用CWindowDC类构造函数



设备坐标系与逻辑坐标系

➤ 设备坐标系

- 单位为像素（设备单位），x轴自左至右，y轴从上到下，坐标原点在屏幕左上角
- 屏幕坐标系：以屏幕左上角为原点，一些与整个屏幕有关的函数均采用屏幕坐标，弹出式菜单使用的也是屏幕坐标
- 窗口坐标系：以窗口左上角为坐标原点，它包括窗口标题栏、菜单栏和工具栏等范围
- 客户区坐标系：以窗口客户区左上角为原点，主要用于客户区的绘图输出和窗口消息的处理
- 屏幕坐标与客户区坐标的相互转换：
`CWnd::ScreenToClient()` `CWnd::ClientToScreen()`

设备坐标系与逻辑坐标系

➤ 设备坐标系

- 单位为像素（设备单位），x轴自左至右，y轴从上到下，坐标原点在屏幕左上角

➤ 逻辑坐标系

- 现实世界所使用的坐标系，面向DC，属于笛卡尔坐标系
- 单位为公制、英制，如毫米、英寸等，又称逻辑单位
- X轴正方向是右，Y轴正方向不定，视映射模式而定
- 在绘图时，Windows会根据当前设置的映射模式将逻辑坐标转换为设备坐标
- 文本和绘图函数使用与客户区坐标对应的逻辑坐标
- 在客户区移动或按下鼠标的鼠标位置是采用客户区设备坐标

移动坐标系原点

➤ 基本概念

- 设备坐标系原点永远在左上角
- 移动的是逻辑坐标系的原点位置，且逻辑坐标系的Y轴正向是可以改变的

➤ 窗口与视口

- 窗口(Window): 对应逻辑坐标系上程序员设定的区域
- 视口(Viewport): 对应实际输出设备上设定的区域

➤ 相关函数

- SetViewportOrg(x,y): 将设备坐标系位置(x,y)设为逻辑坐标系原点
- SetWindowOrg(x,y): 将逻辑坐标系中位置(x,y)设为设备坐标系原点

将原点移到窗口中心点

➤ 使用SetViewportOrg()

OnDraw

```
CRect rect;  
GetClientRect(&rect); //返回的是设备坐标  
pDC->SetViewportOrg(rect.Width()/2,rect.Height()/2);
```

➤ 使用SetWindowOrg()

```
CRect rect;  
GetClientRect(&rect); //返回的是设备坐标  
CPoint point (rect.Width()/2, rect.Height()/2);  
pDC->DPtoLP (&point); // 设备坐标转换为逻辑坐标  
pDC->SetWindowOrg (-point.x, -point.y);
```

映射模式

- 映射模式定义逻辑坐标与设备坐标单位的关系
 - 约束映射模式：比例因子固定
 - 非约束映射模式：由矩形区域推导出比例因子及轴向
- 设置映射模式：CDC的SetMapMode()函数
 - LPtoDP(): 逻辑坐标转为设备坐标
 - DPtoLP(): 设备坐标转为逻辑坐标
- 获得当前映射模式：CDC的GetMapMode()函数

映射模式

映射模式	一个逻辑单位对应的距离	X和Y轴的方向
MM_TEXT	1像素	右、下
MM_LOMETRIC	0.1毫米	右、上
MM_HIMETRIC	0.01毫米	右、上
MM_LOENGLISH	0.01英寸	右、上
MM_HIENGLISH	0.001英寸	右、上
MM_TWIPS	1/1440英寸	右、上
MM_ISOTROPIC	用户定义(x和y同等缩放)	用户自定义
MM_ANISOTROPIC	用户定义(x和y独立缩放)	用户自定义

举例：比较MM_TEXT和MM_LOMETRIC

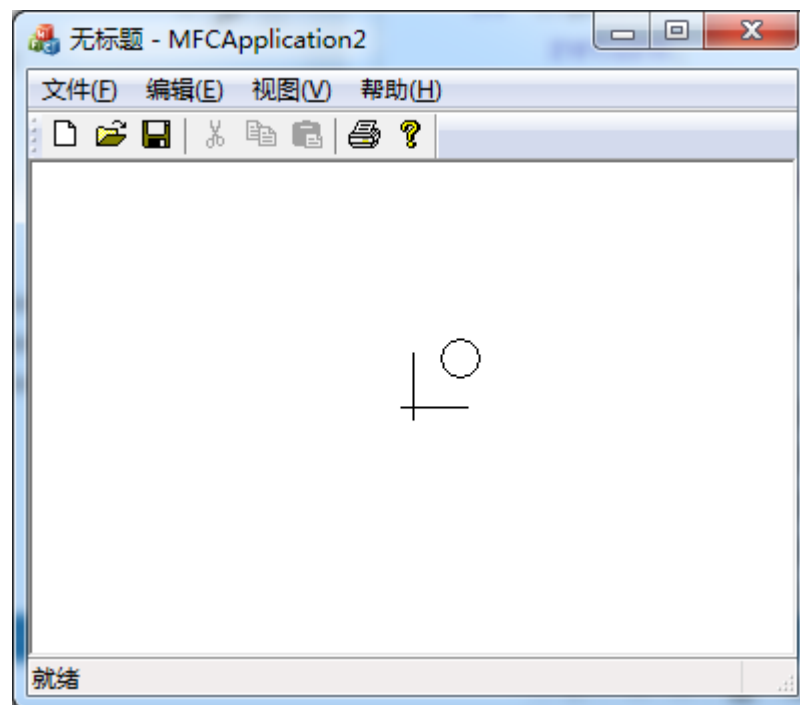
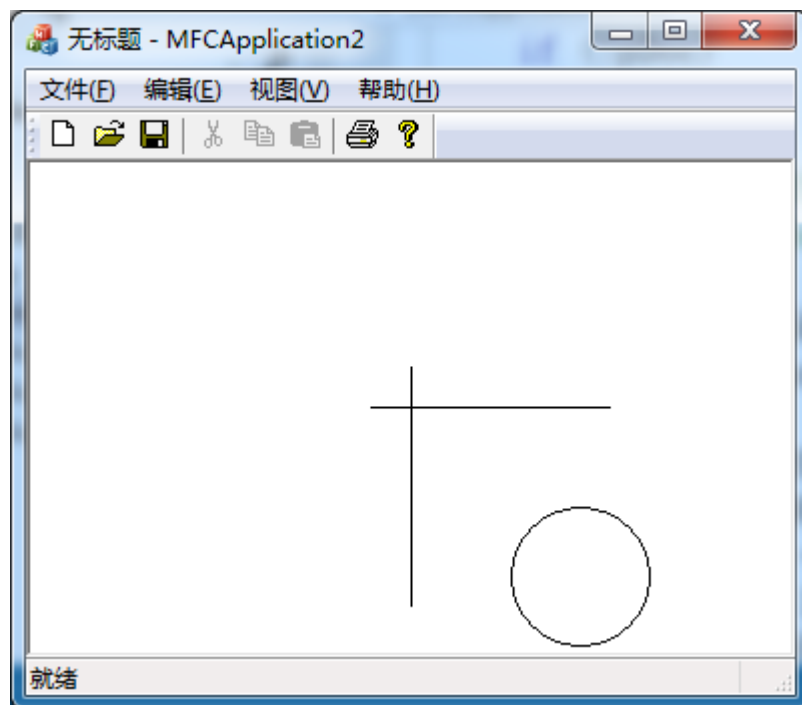


OnDraw

```
// pDC->SetMapMode(MM_LOMETRIC);

//将原点移到窗口中心点
CRect rect;
GetClientRect(&rect);          // 得到客户区域的大小
pDC->SetViewportOrg(rect.Width()/2,rect.Height()/2);
//将设备坐标系位置(x,y)设为逻辑坐标系原点
//绘制直线和圆
pDC->MoveTo(-20,0);
pDC->LineTo(100,0);
pDC->MoveTo(0,-20);
pDC->LineTo(0,100);
pDC->Ellipse(50,50,120,120);
```


举例：比较MM_TEXT和MM_LOMETRIC



OnDraw()与OnPaint()的区别

- OnDraw()是CView类的成员函数，不响应消息
- OnPaint()是CWnd类的成员函数，响应WM_PAINT消息
- CView默认调用的OnPaint()函数调用了OnDraw()函数，一般在OnDraw函数内添加绘图代码，完成绘图任务

系统程序

```
void CView::OnPaint() {  
    CPaintDC dc(this);  
    OnPrepareDC(&dc);  
    OnDraw(&dc);           // 调用OnDraw  
}
```

- Windows系统发送WM_PAINT消息的时机
 - 第一次创建一个窗口时
 - 改变窗口的大小时
 - 把窗口从另一个窗口背后移出时

主要内容

- 主框架窗口与视图窗口
- GDI与设备环境
- 常用绘图函数
- 绘图工具
- 文本处理
- 位图操作
- 图标与光标
- 综合实例

常用绘图函数——直线和曲线

- MoveTo: 在画线前设定当前位置
- LineTo: 从当前位置画一条线到指定位置, 并将当前位置移至线的终点
- Polyline: 将一系列点用线段连接起来
- PolylineTo: 从当前位置开始将一系列点用线段连接起来, 并将当前位置移至折线的终点
- Arc: 画一个弧
- ArcTo: 画一个弧并将当前位置移至弧的终点
- PolyBezier: 画一条或多条贝塞尔样条曲线
- PolyBezierTo: 画一条或多条贝塞尔样条曲线, 并将当前位置移至最后一段样条曲线的终点
- PolyDraw: 通过一组点画一系列线段和贝赛尔样条曲线, 并将当前位置移至最后一个线段或样条曲线的终点

常用绘图函数——封闭图形

- Ellipse: 画一个圆或椭圆
- Chord: 画一个弧并连接两个端点
- Pie: 画一个弧并将两个端点与椭圆中心连接
- Polygon: 连接一组点形成一个多边形
- Rectangle: 画一个带直角的矩形
- RoundRect: 画一个带圆角的矩形

- DrawFocusRect: 用点线画矩形边框
- FillRect: 用画刷填充矩形，不画边框
- FrameRect: 用画刷画矩形边框，不填充内部
- InvertRect: 在矩形区域中反显颜色

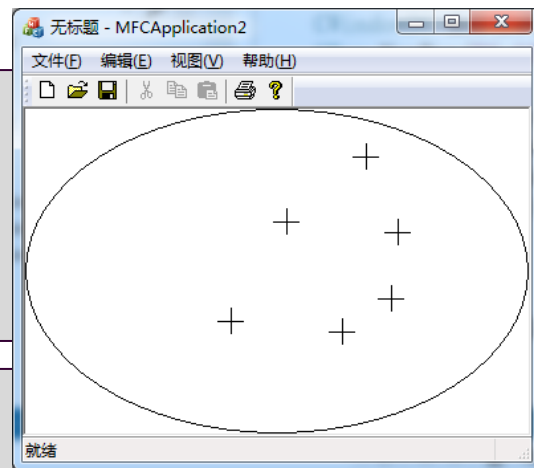
举例：绘制椭圆和直线

- 1) 在客户区画最大的椭圆
- 2) 当点击鼠标左键时，以鼠标左键点击的位置为中心，画长度为20的十字线

OnDraw()函数中添加

```
CRect rect;  
GetClientRect(&rect);  
pDC->Ellipse(rect);
```

```
OnLButtonDown(UINT nFlags, CPoint point) {  
    CClientDC dc(this);  
    dc.MoveTo(point.x-10,point.y); dc.LineTo(point.x+10,point.y);  
    dc.MoveTo(point.x,point.y-10); dc.LineTo(point.x,point.y+10);  
    CView::OnLButtonDown(nFlags, point);  
}
```





课上实验习题

- 阅读书籍 p.56-69
- 练习补充实验材料，并完成下列练习题
- 1，使用MM_TEXT的映射模式。原点位于屏幕客户区中心。绘制坐标系x-y轴
- 2.从起点P0(-100,-50)到终点P1(100,50)绘制一段蓝色直线。
- 3.将客户区矩形上下边界各收缩100个像素绘制矩形